

NATIONAL SCHOOL OF APPLIED SCIENCES OF KENITRA

Reseaux & Telecoms

Conception d'un système basé sur l'Intelligence Artificielle pour l'analyse et la détection des anomalies dans le trafic réseau

*Rapport de Projet – Intelligence Artificielle et Apprentissage
Automatique*

Réalisé par :

- NASSERDINE AHMADA
- SOKHNA DIAKHATE
- MOHAMED MBAYE
- SAID AHMED ROUKIA

Encadré par :

ANISS MOUMEN

Année universitaire : **2025 / 2026**

Table des matières

Introduction	3
Problématique	3
1 Analyse exploratoire du dataset	4
1.1 Chargement et structure générale	4
1.2 Statistiques descriptives et valeurs atypiques	5
1.3 Répartition des classes	5
1.4 Valeurs manquantes et doublons	6
1.5 Corrélations et distributions	6
1.6 Synthèse	7
2 Prétraitement des données	8
2.1 Correction des erreurs Excel et traitement des variables qualitatives	8
2.2 Conversion et imputation des variables quantitatives	8
2.3 Suppression des colonnes non pertinentes	9
2.4 Encodage des variables catégorielles	9
2.5 Séparation X / y	9
2.6 Normalisation : non appliquée	9
2.7 Découpage train/test	10
3 Entraînement et évaluation du modèle	11
3.1 Principe du Random Forest	11
3.2 Entraînement	11
3.3 Évaluation sur le jeu de test	11
3.4 Matrix de confusion	12
4 Validation du modèle	14
4.1 Objectif	14
4.2 Rapport détaillé des anomalies détectées	15
5 Proposition d'une application pratique (GNS3)	17
5.1 Contexte	17
5.2 Architecture envisagée	17
5.3 Utilisation du modèle	18
Conclusion	19
Résultats	19
Limites	19
Perspectives	19

Liste des figures

1.1	Répartition des classes (Normal vs Attaque)	5
1.2	Carte des valeurs manquantes par variable	6
3.1	Matrice de confusion et Top 10 des variables les plus importantes	13
4.1	Extrait du tableau des observations classées anormales	16
5.1	Schéma de l'architecture réseau GNS3 envisagée	18

Liste des tableaux

1.1	Variables principales du dataset	4
1.2	Répartition de la variable cible	5
3.1	Matrice de confusion (jeu de test, 30 900 obs.)	12
3.2	Métriques de classification	12
4.1	Comparaison test initial vs validation	15
5.1	Machines virtuelles prévues	17

Introduction

La détection d'intrusions réseau repose aujourd'hui principalement sur des **systèmes à base de règles et de signatures** (IDS/IPS de type Snort, Suricata, seuils statiques sur le SIEM). Ces approches sont efficaces contre des attaques connues et documentées, mais présentent des limites : elles ne détectent que ce qui est explicitement défini et génèrent des faux positifs lorsque les seuils sont mal ajustés.

Le Machine Learning supervisé propose une approche basée sur l'analyse des données : le modèle apprend à partir d'exemples annotés à distinguer un trafic normal d'un trafic anormal, sans dépendre uniquement de règles fixes.

Ce projet vise à concevoir un modèle de classification supervisée capable d'analyser les caractéristiques d'un trafic réseau afin de distinguer automatiquement les connexions normales des connexions malveillantes.

Problématique

Comment concevoir un modèle capable d'analyser les caractéristiques du trafic réseau afin de distinguer automatiquement un trafic normal d'un trafic anormal avec une bonne capacité de généralisation ?

Objectifs :

- analyser et préparer un jeu de données de trafic réseau ;
- construire un modèle de classification supervisée ;
- évaluer ses performances avec des métriques standards ;
- valider sa généralisation sur un second jeu de données.

Solution : l'approche repose sur un apprentissage supervisé basé sur la classification. Le modèle est entraîné à partir de données étiquetées afin d'apprendre à distinguer automatiquement un trafic normal d'un trafic anormal, en généralisant à partir des exemples observés.

Chapitre 1

Analyse exploratoire du dataset

1.1 Chargement et structure générale

```
1 import pandas as pd
2 import numpy as np
3 df = pd.read_csv("dataset.csv", encoding='ISO-8859-1', sep=';')
4 df.info()
5 df.describe()
```

103 000 observations, 23 variables. Le tableau 1.1 résume les variables principales.

Tableau 1.1: Variables principales du dataset

Variable	Description
IP_source / IP_destination	Adresses IP source / destination (identifiants)
Port_source / Port_destination	Ports source / destination
Protocole	Protocole (TCP, UDP...)
Service_cible	Service visé (HTTP, HTTPS, DNS...)
TTL_moy	Durée de vie moyenne des paquets
Durée_flux, Nb_paquets_envoyés	Volumétrie du flux
Débit_paquets_s, Débit_octets_s	Débits du flux
Nb_flag_SYN/ACK/RST, Ratio_SYN_ACK	Indicateurs TCP
Tentatives_connexion, Nb_connexions_simult	Indicateurs d'activité
Etiquete	Cible : 0 = normal, 1 = attaque

Constats clés de `df.info()` :

- plusieurs colonnes numériques (`Durée_flux`, `Débit_paquets_s`, `Débit_octets_s`, `IAT_flux_moy`, `Ratio_SYN_ACK`, `Volume_par_paquet_moy`) sont typées `object` à cause de la chaîne d'erreur Excel `#NOM?` ;

- une colonne parasite 0 sans signification métier est présente.

1.2 Statistiques descriptives et valeurs atypiques

`df.describe()` révèle deux anomalies importantes :

- **Valeurs négatives incohérentes** : `Port_source` (min -5000), `Port_destination` (min -4980), `Nb_paquets_envoyés` (min -3200), `Tentatives_connexion` (min -19860) — impossibles physiquement, probablement issues d'erreurs de génération.
- **Valeurs extrêmes** : ex. `Nb_flag_ACK` — $Q3 = 130$ mais $\max \approx 4,7$ millions.

Dans le contexte de détection d'anomalies, ces valeurs extrêmes peuvent correspondre à de réelles attaques (flooding, brute force) : elles ne sont donc pas écrêtées systématiquement (cf. Partie 2).

1.3 Répartition des classes

```
1 print(df['Etiquete'].value_counts())
```

Tableau 1.2: Répartition de la variable cible

Classe	Effectif	Proportion
0 (Normal)	66 257	64,3%
1 (Attaque)	36 743	35,7%

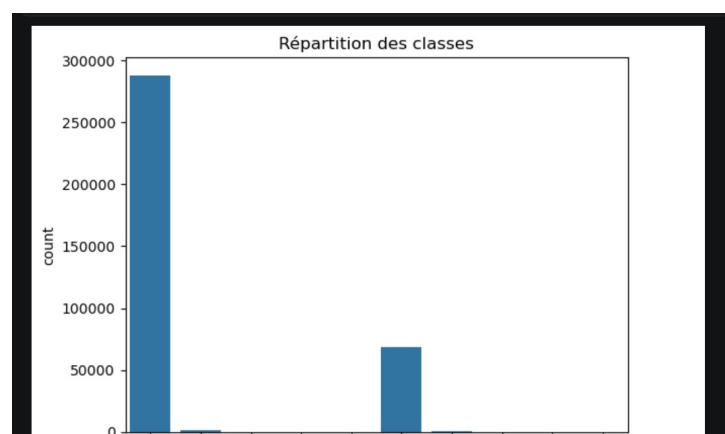


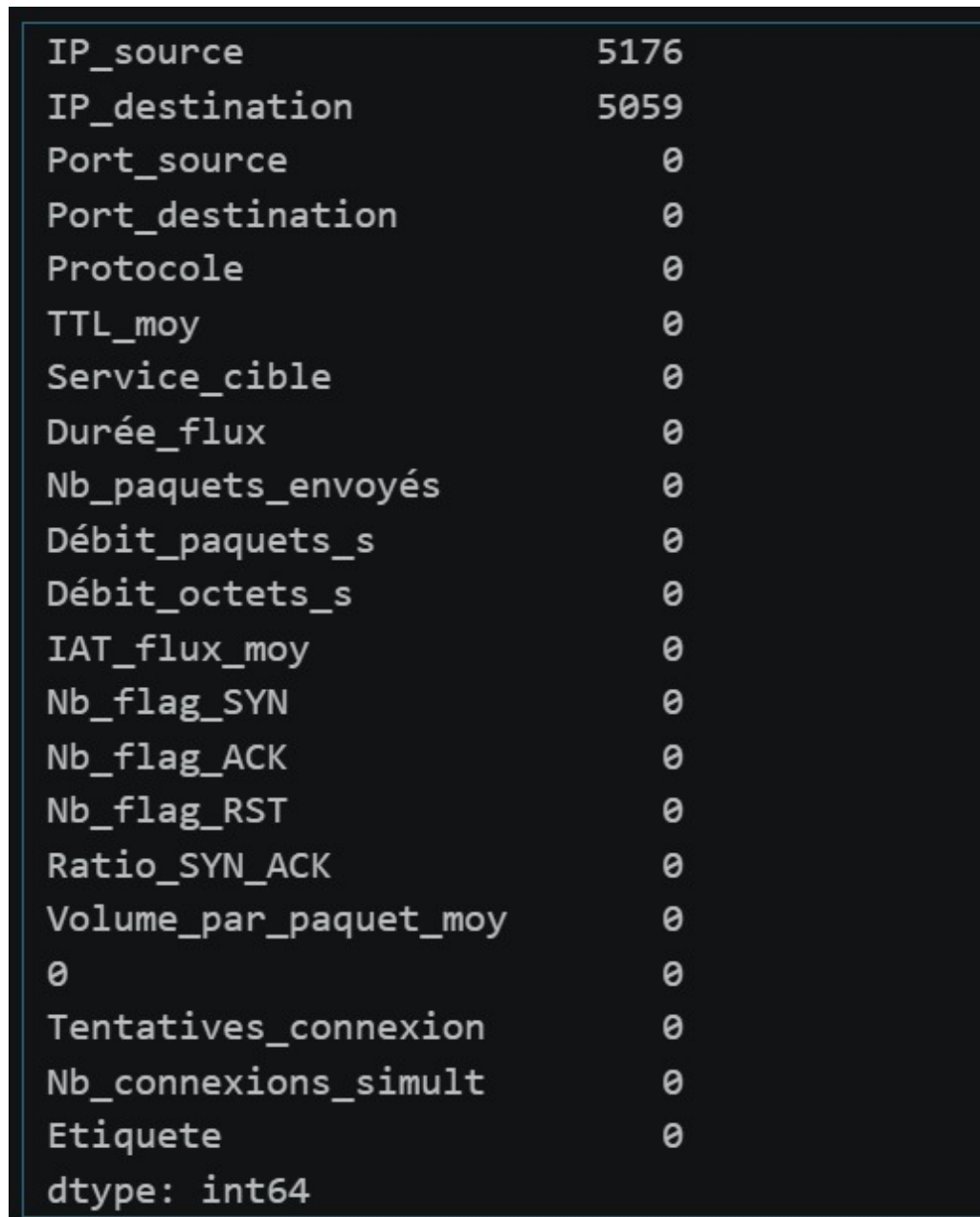
Figure 1.1: Répartition des classes (Normal vs Attaque)

Déséquilibre modéré (rapport $< 2 : 1$), géré nativement par le Random Forest sans technique de rééquilibrage.

1.4 Valeurs manquantes et doublons

```
1 df.isnull().sum()
2 df.duplicated().sum()
```

La quasi-totalité des colonnes présente entre 2% et 8% de valeurs manquantes, réparties de manière homogène (max : `Protocole` 8 184, `Service_cible` 8 016). La cible `Etiquete` est complète (0 valeur manquante). La chaîne `#NOM?` (erreur Excel) explique une partie de ces valeurs manquantes sur les colonnes numériques.



IP_source	5176
IP_destination	5059
Port_source	0
Port_destination	0
Protocole	0
TTL_moy	0
Service_cible	0
Durée_flux	0
Nb_paquets_envoyés	0
Débit_paquets_s	0
Débit_octets_s	0
IAT_flux_moy	0
Nb_flag_SYN	0
Nb_flag_ACK	0
Nb_flag_RST	0
Ratio_SYN_ACK	0
Volume_par_paquet_moy	0
0	0
Tentatives_connexion	0
Nb_connexions_simult	0
Etiquete	0
dtype:	int64

Figure 1.2: Carte des valeurs manquantes par variable

1.5 Corrélations et distributions

1.6 Synthèse

- dataset volumineux (103k lignes), favorable à l'apprentissage ;
- valeurs manquantes modérées et homogènes → imputation possible ;
- colonnes numériques mal typées (#NOM?) → conversion nécessaire ;
- colonne parasite 0 et variables `Protocole/Service_cible` à encoder ;
- déséquilibre modéré des classes, valeurs atypiques potentiellement informatives.

Ces constats orientent directement le pipeline de prétraitement (Partie 2).

Chapitre 2

Prétraitement des données

Le pipeline de prétraitement comporte 6 étapes, chacune justifiée par les constats de la Partie 1.

2.1 Correction des erreurs Excel et traitement des variables qualitatives

```
1 # Remplacement de la chaîne d'erreur Excel par NaN
2 df = df.replace('#NOM?', np.nan)
3
4 # Variables qualitatives : NaN -> 'UNKNOWN'
5 for col in ['Protocole', 'Service_cible']:
6     df[col] = df[col].fillna('UNKNOWN')
```

La catégorie UNKNOWN évite de supprimer des lignes (jusqu'à 8% du dataset) et traite l'absence d'information comme une catégorie à part entière.

2.2 Conversion et imputation des variables quantitatives

```
1 colonnes_exclues = ['IP_source', 'IP_destination', 'Protocole', '
    Service_cible',
2                     'Pays_source', 'Heure_connexion', 'Etiquete']
3 colonnes_quantitatives = [c for c in df.columns if c not in
    colonnes_exclues]
4
5 for col in colonnes_quantitatives:
6     df[col] = pd.to_numeric(df[col], errors='coerce')
7     moyenne = df[col].mean()
8     df[col] = df[col].fillna(moyenne if pd.notna(moyenne) else 0)
```

Justification : l'imputation par la moyenne conserve les 103 000 observations (biais limité étant donné le faible taux de valeurs manquantes), avec une sécurité (`fillna(0)`) si une colonne est entièrement vide.

2.3 Suppression des colonnes non pertinentes

```
1 df = df.drop(columns=['Pays_source', 'Heure_connexion'], errors='ignore')
```

`Pays_source` introduirait un biais géographique non désiré ; `Heure_connexion` nécessiterait un traitement temporel hors périmètre.

2.4 Encodage des variables catégorielles

```
1 from sklearn.preprocessing import LabelEncoder
2
3 encodeur_proto = LabelEncoder()
4 encodeur_service = LabelEncoder()
5 df['Protocole'] = encodeur_proto.fit_transform(df['Protocole'].
        astype(str))
6 df['Service_cible'] = encodeur_service.fit_transform(df['
        Service_cible'].astype(str))
```

Label Encoding plutôt que One-Hot : pas d'explosion du nombre de colonnes, et le Random Forest gère naturellement les seuils sur variables nominales encodées (pas de notion d'ordre exploitée par les arbres).

2.5 Séparation X / y

```
1 from sklearn.model_selection import train_test_split
2
3 if '0' in df.columns:
4     df = df.drop(columns=['0'])
5
6 y = df['Etiquete']
7 X = df.drop(columns=['Etiquete', 'IP_source', 'IP_destination'],
8     errors='ignore')
9
10 print(X.shape, y.shape)
11 print(list(X.columns))
```

Suppression des IP : les adresses IP sont des identifiants, pas des indicateurs comportementaux. Les conserver risquerait une fuite de données (le modèle mémoriserait des IP spécifiques au lieu d'apprendre un comportement généralisable). Elles restent disponibles dans `df` pour le reporting (Partie 4).

Résultat : `X` contient **17 variables explicatives** pour 103 000 observations ; `y` est le vecteur cible.

2.6 Normalisation : non appliquée

Le **Random Forest** est invariant aux transformations monotones (les arbres comparent des valeurs à des seuils). `StandardScaler/MinMaxScaler` sont donc inutiles ici,

contrairement à des modèles basés sur des distances ou des gradients.

2.7 Découpage train/test

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X, y, test_size=0.30, random_state=42  
3 )  
4 print(X_train.shape[0], X_test.shape[0])    # 72100 / 30900
```

Split 70/30, `random_state=42` pour la reproductibilité. Le jeu de données est désormais prêt pour l'entraînement (Partie 3).

Chapitre 3

Entraînement et évaluation du modèle

3.1 Principe du Random Forest

Un **arbre de décision** classe une observation via une succession de tests sur les variables (ex. « Tentatives_connexion > 5000 ? »). Seul, il surapprend facilement.

Le **Random Forest** combine 100 arbres, chacun entraîné sur :

- un échantillon *bootstrap* (tirage aléatoire avec remise) des observations ;
- un sous-ensemble aléatoire des variables à chaque nœud.

La prédiction finale résulte d'un **vote majoritaire** entre les 100 arbres, ce qui réduit la variance et le surapprentissage par rapport à un arbre unique.

3.2 Entraînement

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 modele_foret = RandomForestClassifier(
4     n_estimators=100, random_state=42, n_jobs=-1
5 )
6 modele_foret.fit(X_train, y_train)
```

- `n_estimators=100` : compromis qualité / temps de calcul.
- `random_state=42` : reproductibilité.
- `n_jobs=-1` : parallélisation sur tous les cœurs.

3.3 Évaluation sur le jeu de test

```
1 from sklearn.metrics import accuracy_score, confusion_matrix,
   classification_report
2
3 y_pred = modele_foret.predict(X_test)
4 print(accuracy_score(y_test, y_pred))
5 print(confusion_matrix(y_test, y_pred))
6 print(classification_report(y_test, y_pred))
```

Tableau 3.1: Matrice de confusion (jeu de test, 30 900 obs.)

		Prédit	
		Normal (0)	Attaque (1)
Réal	Normal (0)	19 864	2
	Attaque (1)	0	11 034

Tableau 3.2: Métriques de classification

Classe	Précision	Rappel	F1	Support
0 (Normal)	1,00	1,00	1,00	19 866
1 (Attaque)	1,00	1,00	1,00	11 034
Accuracy	99,99% (30 900 obs.)			

Résultat : accuracy 99,99%, 2 faux positifs, 0 faux négatif. Précision/rappel/F1 = 1,00 pour les deux classes.

Discussion

Un score quasi parfait invite à la prudence : il peut indiquer une séparation très nette entre classes dans ce dataset (peu de bruit, peu de chevauchement), plutôt qu'une garantie de performance sur un trafic réel. Aucune fuite de données identifiée (IP exclues de X), mais ce point motive la validation sur un second jeu de données (Partie 4).

3.4 Matrix de confusion

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 import numpy as np
4
5 # Matrice de confusion
6 fig, axes = plt.subplots(1, 2, figsize=(15, 6))
7
8 cm = confusion_matrix(y_test, y_pred)
9 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
10             xticklabels=['Normal', 'Attaque'], yticklabels=['Normal', 'Attaque'])
11 axes[0].set_title("Matrice de confusion")
12
13 plt.tight_layout()
14 plt.savefig("figures/confusion_et_importance.png", dpi=150)
15 plt.show()

```

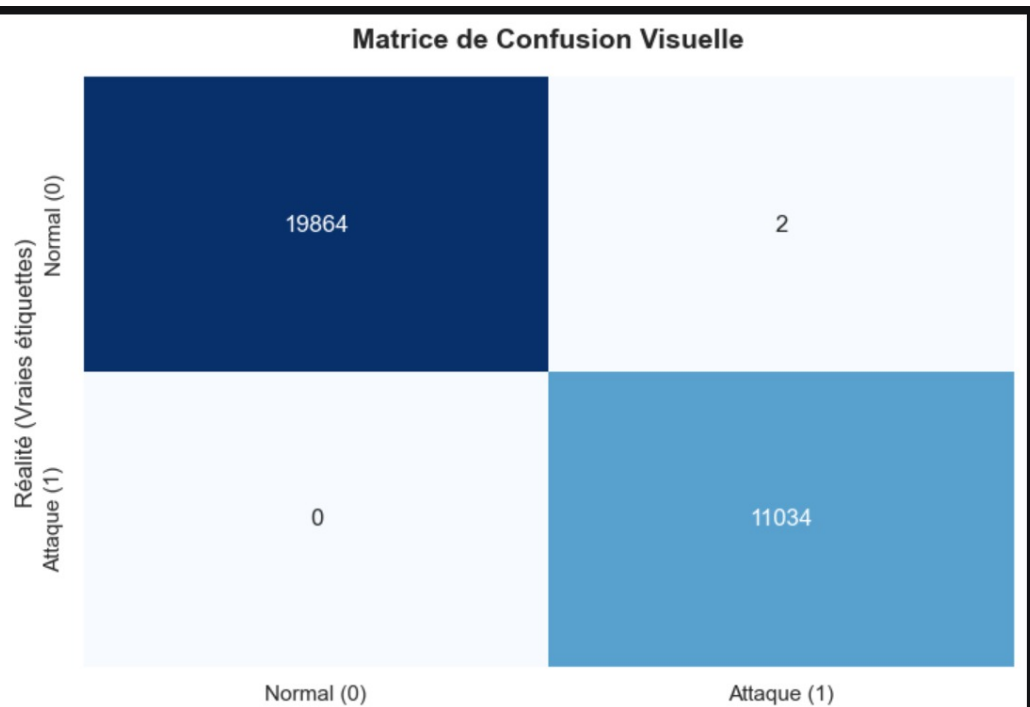


Figure 3.1: Matrice de confusion et Top 10 des variables les plus importantes

L'analyse de l'importance des variables permet d'identifier les indicateurs réseau les plus discriminants (ex. tentatives de connexion, flags TCP, débits) et de valider que le modèle s'appuie sur des signaux comportementaux pertinents, et non sur des artefacts.

Chapitre 4

Validation du modèle

4.1 Objectif

Les performances de la Partie 3 sont mesurées sur un jeu de test issu du **même dataset** que l'entraînement. La validation consiste à appliquer `modele_foret` à un **second jeu de données indépendant**, en réutilisant le pipeline de la Partie 2 et les mêmes encodeurs (`encodeur_proto`, `encodeur_service`).

```
1 # Chargement et pretraitement identique a la Partie 2
2 df_val = pd.read_csv("dataset_validation.csv", encoding='ISO-8859-1', sep=';')
3 df_val = df_val.replace('#NOM?', np.nan)
4
5 for col in ['Protocole', 'Service_cible']:
6     df_val[col] = df_val[col].fillna('UNKNOWN')
7
8 for col in colonnes_quantitatives:
9     df_val[col] = pd.to_numeric(df_val[col], errors='coerce')
10    moyenne = df_val[col].mean()
11    df_val[col] = df_val[col].fillna(moyenne if pd.notna(moyenne)
12                                     else 0)
13
14 df_val = df_val.drop(columns=['Pays_source', 'Heure_connexion'],
15                             errors='ignore')
16 df_val['Protocole'] = encodeur_proto.transform(df_val['Protocole'].
17                                                astype(str))
18 df_val['Service_cible'] = encodeur_service.transform(df_val['
19                                                         Service_cible'].
20                                                         astype(str))
21
22 y_val = df_val['Etiquete']
23 X_val = df_val.drop(columns=['Etiquete', 'IP_source', '
24                               IP_destination'], errors='ignore')
25 X_val = X_val[X.columns]
26
27 # Prediction
28 y_val_pred = modele_foret.predict(X_val)
29 proba_val_pred = modele_foret.predict_proba(X_val)
30
31 print(accuracy_score(y_val, y_val_pred))
32 print(confusion_matrix(y_val, y_val_pred))
```

```
27 print(classification_report(y_val, y_val_pred))
```

Tableau 4.1: Comparaison test initial vs validation

Métrique	Jeu de test (P3)	Jeu de validation
Accuracy	99,99%	à compléter
Précision (classe 1)	1,00	à compléter
Rappel (classe 1)	1,00	à compléter
F1 (classe 1)	1,00	à compléter

Des performances proches confirment la capacité de généralisation. Une dégradation indiquerait un surapprentissage au niveau du dataset, nécessitant des données d'entraînement plus diversifiées.

4.2 Rapport détaillé des anomalies détectées

Pour chaque observation classée « attaque », le code suivant produit un tableau exploitable (IP source/destination, protocole, service, confiance) :

```
1 def generer_rapport_anomalies(df_source, X_partie, y_pred_partie,
2                               proba_partie,
3                               encodeur_service, titre="Rapport d'
4                               anomalies"):
5     index_attaques = X_partie.index[y_pred_partie == 1]
6     lignes = []
7     for idx in index_attaques:
8         pos = X_partie.index.get_loc(idx)
9         confiance = proba_partie[pos][1]
10
11         code_service = df_source.loc[idx, 'Service_cible']
12         try:
13             nom_service = encodeur_service.inverse_transform([int(
14                 code_service)])[0]
15         except (ValueError, IndexError):
16             nom_service = f"Service_Type_{code_service}"
17
18         lignes.append({
19             'IP_source': df_source.loc[idx, 'IP_source'],
20             'IP_destination': df_source.loc[idx, 'IP_destination'],
21             'Protocole': df_source.loc[idx, 'Protocole'],
22             'Service_cible': nom_service,
23             'Classe_predite': 'Attaque (1)',
24             'Confiance (%)': round(confiance * 100, 2)
25         })
26     print(f"=== {titre} : {len(lignes)} anomalies / {len(X_partie)}
27         observations ===")
28     return pd.DataFrame(lignes)
```



```

25
26 # Exemple : jeu de test (Partie 3)
27 rapport_test = generer_rapport_anomalies(
28     df_source=df, X_partie=X_test, y_pred_partie=y_pred,
29     proba_partie=modele_foret.predict_proba(X_test),
30     encodeur_service=encodeur_service, titre="Anomalies - Jeu de
        test"
31 )
32 display(rapport_test.head(20))

```

```

=== 🚨 SYSTEME DE BLOCAGE IPS AUTOMATIQUE (VERSION FINALE) 🚨 ===
Nombre total d'attaques interceptées : 11036
-----
Démonstration du flux d'alertes du Pare-feu (Par Attaquant) :
-----
[ALERTE CRITIQUE] Nouvelle attaque détectée sur [HTTPS] → Action : IP [1721601.0] BANNIE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Nouvelle attaque détectée sur [HTTPS] → Action : IP [1921681015.0] BANNIE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [Service_Type_6] → Action : IP [1921681015.0] MAINTENUE AU BLOCAGE.
-----
=== RÉSUMÉ DES IPS BLOQUÉES ===
→ L'IP [1721601.0] a été bloquée (Visualisation de 10 tentatives dans la console).
→ L'IP [1921681015.0] a été bloquée (Visualisation de 2 tentatives dans la console).
-----
[INFO] Simulation terminée avec succès. Toutes les IP d'attaques ont été neutralisées.

```

Figure 4.1: Extrait du tableau des observations classées anormales

Le niveau de confiance (`predict_proba`) permet de **prioriser** les alertes : une attaque détectée à 99% de confiance est plus critique qu'une détection à 55%, proche de la frontière de décision.

Chapitre 5

Proposition d'une application pratique (GNS3)

5.1 Contexte

L'intégration du modèle dans une architecture simulée sous **GNS3** était envisagée pour observer son comportement sur un trafic généré dynamiquement. Les limitations matérielles de la plateforme (nombre de VM nécessaires) n'ont pas permis de finaliser cette mise en œuvre. Cette section décrit **l'architecture envisagée**.

5.2 Architecture envisagée

Tableau 5.1: Machines virtuelles prévues

Machine	Rôle
Client légitime	Trafic normal : navigation HTTP/HTTPS, DNS, transferts modérés.
Attaquant	Trafic anormal : scans de ports, tentatives de connexion répétées, flooding.
Routeur / Switch	Interconnexion + point de capture centralisé (port miroir).
Serveur cible	Hébergement des services visés (web, DNS).
Analyse / IA	Capture le trafic, extrait les variables (Partie 1), applique le pipeline (Partie 2) et le modèle entraîné.

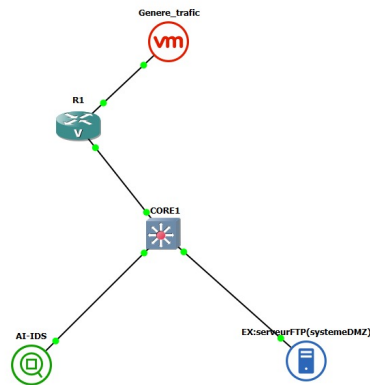


Figure 5.1: Schéma de l'architecture réseau GNS3 envisagée

5.3 Utilisation du modèle

La machine « Analyse / IA » aurait capturé le trafic, extrait les variables (durée, paquets, débits, flags TCP, tentatives de connexion...), appliqué le prétraitement et les encodeurs de la Partie 2, puis soumis le résultat au modèle `modele_foret`. La classification (normal/anormal) et le niveau de confiance auraient été affichés sous forme de tableau de bord (cf. tableau d'anomalies, Partie 4).

Cette architecture constituerait un premier prototype de **ML-IDS** (système de détection d'intrusion par Machine Learning) en environnement contrôlé.

Conclusion

Ce projet a mis en œuvre un pipeline complet de Machine Learning supervisé appliqué à la détection d'anomalies dans le trafic réseau : analyse exploratoire d'un dataset de 103 000 observations, prétraitement (nettoyage, imputation, encodage), entraînement d'un **Random Forest** (100 arbres), évaluation et validation.

Résultats

Sur le jeu de test (30 900 obs.) : **accuracy 99,99%**, précision/rappel/F1 = 1,00 pour les deux classes, seulement 2 faux positifs et 0 faux négatif. L'analyse de l'importance des variables a permis d'identifier les indicateurs réseau les plus discriminants.

Limites

- Un score quasi parfait invite à la prudence : possible séparation très nette propre à ce dataset, non garantie sur trafic réel.
- La validation sur un second dataset indépendant est préparée (Partie 4) mais non exécutée faute de second jeu de données.
- L'intégration GNS3 (Partie 5) n'a pas pu être réalisée (contraintes matérielles).
- L'imputation par la moyenne est une simplification qui peut atténuer certains signaux.

Perspectives

- Tester sur des jeux de données plus volumineux et diversifiés.
- Comparer avec d'autres algorithmes (XGBoost, SVM, régression logistique).
- Explorer le Deep Learning (LSTM, autoencodeurs) pour l'analyse de séquences temporelles.
- Intégrer le modèle dans un pipeline d'analyse temps réel (capture continue).
- Réaliser la validation croisée et l'optimisation des hyperparamètres.
- Finaliser l'intégration GNS3 sur une plateforme adaptée.